

UADE

Exposición de experto

Programación III

Temas a desarrollar

1

Introducción a la Materia

2

Conceptos Básicos

3

Complejidad Temporal Alg. Recursivos

4

Divide y Conquista

5

Búsqueda Binaria

6

Palabra Palíndromo

7

Función de Fibonacci

8

Métodos de Ordenamiento



Introducción a la Materia



Bibliografía



BRASSARD, Gilles. *Fundamentals of algorithmics*. Upper Saddle River: Prentice Hall, 1996. ISBN: 9780133350685

Qué es Programación III – Diseño y Análisis de Algoritmo

- ❑ Estudio de diferentes técnicas de diseño de algoritmos, con el objetivo de encontrar la solución más adecuada desde el punto de vista de eficiencia temporal a distintos tipos de problemas.
- ❑ Los algoritmos son independientes de los lenguajes de programación que se utilicen para codificar la solución diseñada.
- ❑ El análisis de la eficiencia temporal es uno de los puntos más importantes para evaluar el algoritmo diseñado.

Información de Aprobación y Fechas Importantes

- ☐ Aprobación: se deben aprobar los dos parciales con 4 o más, con una instancia de recuperatorio, y el Final en instancia regular o previa.
- ☐ Primer Parcial: **04/05/2026 (13:30 hrs.)**
- ☐ Segundo Parcial: **07/05/2026 (13:30 hrs.)**
- ☐ Recuperatorio: **08/05/2026 (13:30 hrs.)**
- ☐ Final Regular: **09/05/2026 (14:00 hrs.)**

Por qué Algoritmos



Por qué Pseudocódigo





Conceptos básicos de algoritmos



Conocimientos Esperados

Se espera que el alumno conozca los siguientes conceptos de las materias previamente cursadas:

- Se trabajaran con tipos de datos primitivos (enteros, char, decimal, etc.)
- Se trabajaran con estructuras de acceso directo (arreglos, vectores, etc.).
- Se espera que conozcan recorridos sobre vectores, matrices, etc.
- Se espera que el alumno tenga conocimientos matemáticos básicos de matrices, funciones de una, dos o más incógnitas y generación de gráficos de dos dimensiones.

Conceptos básicos – Programa/Instancia

- **Programa:** es la escritura de la estrategia de un algoritmo en un lenguaje de programación en particular (Java, C++, Delphi, etc.) para ser ejecutado en una computadora.
- **Instancia:** una instancia de un problema es una combinación válida de los posibles datos de entrada. Un problema puede tener una o más instancias, incluso infinitas.
- Un algoritmo será **correcto o eficaz** si funciona de la forma esperada para todas las instancias de un problema.

Conceptos básicos – Eficiencia/Correctitud

- Un algoritmo será **eficiente** si resuelve un problema utilizando una cantidad acotada de recursos y tiempo.
- La algoritmia se ocupa, entonces, del estudio de los algoritmos y abarca el análisis de la **correctitud** y de la **eficiencia**. Es decir, analiza si un algoritmo resuelve un problema y la cantidad de recursos necesarios para llegar a la solución.

Conceptos básicos – Eficiencia temporal

- **Medición práctica:** a través de la medición del tiempo que lleva obtener una solución para una entrada de datos dada. Para poder llevar a cabo esta medición se requiere tener un programa del algoritmo.
- **Medición teórica:** se trata de medir la cantidad de operaciones elementales que ejecuta para un tamaño de datos de entrada.
- La eficiencia nos permite comparar diferentes algoritmos/programas entre sí en cuanto a los usos de recursos.
- No debe depender del *hardware* ni del lenguaje de programación.

Conceptos básicos – Eficiencia temporal

- Principio de invarianza

Dado un algoritmo A y dos implementaciones del mismo I_1 e I_2 , donde la primera tarda $T_1(n)$ y la segunda $T_2(n)$ segundos, entonces existen un número real $c > 0$ y un entero n_0 tal que $T_1(n) \leq cT_2(n)$ para todo $n > n_0$.

- Esto significa que dos implementaciones distintas de un mismo algoritmo no difieren entre sí en más de una constante multiplicativa.

Cálculo eficiencia

- **Cálculo del número de operaciones elementales**
 - Se considera **operación elemental constante** a las asignaciones, operaciones aritméticas o lógicas, acceso a la posición de un vector, llamada a un método (en este caso se deberá analizar la complejidad del método). En todos estos casos suma una constante **1 (UNO)**.
 - Para **una secuencia de operaciones**, la cantidad de operaciones elementales es la suma de la cantidad de operaciones elementales de cada parte.

Cálculo eficiencia

- **Cálculo del número de operaciones elementales**
 - Para un *condicional*, la cantidad de operaciones elementales es el costo de la evaluación de la condición más el máximo entre la cantidad de operaciones de las dos ramas de la condición.
 - Para un *ciclo*, la cantidad de operaciones elementales es N veces la cantidad de operaciones elementales del bloque de repetición, donde N es la cantidad de veces que se repite el ciclo.
 - Para una *llamada a un método*, el número de operaciones elementales es 1 más el número de operaciones elementales de la ejecución del método.

Cálculo eficiencia – tipos de análisis

- **Peor caso:** es el mayor de los costos de todas las instancias de un problema.
- **Caso promedio:** es el promedio de los costos de todas las instancias de un problema.
- **Mejor caso:** es el menor de los costos de todas las instancias de un problema.

Nos centraremos en el análisis del peor caso ya que es el que aporta una cota superior de todos los demás casos.

Notación asintótica

$f(n) = 1$	constante
$f(n) = \log(n)$	logarítmica
$f(n) = n$	lineal
$f(n) = n \log(n)$	
$f(n) = n^2$	cuadrática
$f(n) = n^k$	polinomial
$f(n) = 2^n$	exponencial
$f(n) = b^n$	exponencial
$f(n) = n!$	factorial



Complejidad en algoritmos recursivos



Resolución de recurrencia

- Resolución de funciones/procedimientos recursivos: cálculo de la función factorial de n.

```
int factorial(int n){  
    if(n == 1){  
        return 1;  
    }else{  
        return n*factorial(n-1);  
    }  
}
```

Resolución de recurrencia - Métodos

- **Sustracción**: la llamada recursiva es del tipo $T(n-b)$, donde la función es del tipo

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$

Donde a es la cantidad de llamadas recursivas en el peor de los casos. b es la cantidad de unidades en que disminuye la entrada en cada llamado recursivo. k es el grado del polinomio $p(n)$ de aquellas sentencias a ejecutar fuera del llamado recursivo. Entonces se llega a:

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

Solución sustracción

```
int factorial(int n){  
    if(n == 1){  
        return 1;  
    }else{  
        return n*factorial(n-1);  
    }  
}
```

El factorial es del tipo sustracción $T(n-b)$, dónde $a=1$, $b=1$ y $k=0$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

Como $a=1$ se da la opción $\Theta(n^{k+1})$, y al ser $k=0$, la complejidad es:

$\Theta(n)$

Resolución de recurrencia - Métodos

- **División:** la llamada recursiva es del tipo $T(n/b)$, donde la función es del tipo

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n/b) + f(n) & \text{si } n \geq b \end{cases}$$

Donde a es la cantidad de llamadas recursivas en el peor de los casos. b es la cantidad de unidades en que se divide la entrada en cada llamado recursivo. k es el grado del polinomio $f(n)$ de aquellas sentencias a ejecutar fuera del llamado recursivo. Entonces se llega a:

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$

Caso División

```
public long exp2(long a, long b) {  
    if ( b == 0 ) {  
        return 1;  
    } else {  
        long result = exp2(a*a, b/2);  
        if ( b%2 == 1 ) {  
            result = a * result;  
        }  
        return result;  
    }  
}
```

El factorial es del tipo división $T(n/b)$, donde $a=1$, $b=2$ y $k=0$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$

Como $a=b^k$ la opción es $\Theta(n^k \log(n))$,
como $k=0$:

$\Theta(\log(n))$

Donde n depende de b



Ejercicios a Resolver



Cálculos de la complejidad temporal

- Complejidad temporal: funciones recursivas

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$



$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n/b) + f(n) & \text{si } n \geq b \end{cases}$$



$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$

Ejercicios de aplicabilidad de complejidad temporal

ALGORITMO AlgoritmoA

Entrada: n

bool a = obtenerValor(n)

si (n = 1)

devolver a

fin si

sino

si (a = verdadero)

imprimir (n)

AlgoritmoA(n/2)

fin si

sino

AlgoritmoA(n/2)

fin sino

fin sino

fin Algoritmo A

nota: la complejidad de la función "obtenerValor" es $\Theta(n^2)$, qué valor debería tener imprimir?

a = 1

b = 2

$P(O(n^2)) \rightarrow k = 2$

$a > b^k \rightarrow a > b^2 \quad 1 < 4 \rightarrow O(n^k) \rightarrow O(n^2)$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$

Si imprimir(n) fuese $O(n) \rightarrow$

$P(n) = O(n^2) + O(n)$

Sí influye si imprimir es mayor a $O(n^2)$

Ejercicios de aplicabilidad de complejidad temporal

ALGORITMO AlgoritmoB

Entrada: n

bool a = obtenerValor(n)

si (n = 1)

devolver a

fin si

sino

si (a = verdadero)

calcularCosto (n)

AlgoritmoB(n-2)

fin si

sino

AlgoritmoB(n-2)

fin sino

fin sino

fin Algoritmo B

nota: la complejidad de la función "obtenerValor" es $\Theta(C)$ y "calcularCosto" es $\Theta(n^2)$

a = 1

b = 2

$P(O(n^2)) \rightarrow k = 2$

$a \neq 1 \rightarrow 1=1 \rightarrow O(n^{k+1}) \rightarrow$

$O(n^{2+1}) \rightarrow O(n^3)$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

Ejercicios de aplicabilidad de complejidad temporal

ALGORITMO AlgoritmoC

Entrada: n

bool a = obtenerValor(n)

si (n = 1)

devolver a

fin si

sino

entero c = 1

mientras (c <= 3)

AlgoritmoC(n-2)

c ← c+1

fin mientras

fin sino

fin Algoritmo C

nota: la complejidad de la función “obtenerValor” es $\Theta(C)$

a = 3 porque itera el ciclo en donde esta el llamado recursivo 3 veces

b = 2

$P(O(C)) \rightarrow k = 0$

$a \neq 1 \rightarrow 3 > 1 \rightarrow O(a^{n \text{ div } b}) \rightarrow O(3^{n \text{ div } 2}) \rightarrow O(3^{n/2})$ complejidad exponencial

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$



Divide y Conquista





Divide y Conquista

- **Divide y Conquista** es una técnica de diseño de algoritmos que consiste en dividir un problema “*grande*” en problemas más “*pequeños*”, de resolución más sencilla, y luego combinar las soluciones de los problemas más “*pequeños*” para hallar la solución del problema “*grande*”.
- El problema original se divide en problemas más pequeños de la misma naturaleza hasta llegar a un problema lo suficientemente “*pequeño*” de resolución en “**forma trivial**”. Ésta división se va realizando en forma recursiva.

Divide y Conquista – Estrategia de Solución

Casos bases o triviales

Son aquellos casos que el tamaño de problema tiene solución directa. Podría ser que el caso base sea solo uno.

Subproblemas del mismo tipo

Se debe identificar cómo se va a ir particionando la entrada en tamaños menores, para ir resolviendo cada uno de esos subproblemas por separado en forma independiente.

Combinación de subproblemas

Definir cómo se unificarán los resultados de los subproblemas para dar la solución al problema de tamaño n , es decir al original para el cual se estaba buscando la solución

Divide y Conquista – Esquema General de Solución

ALGORITMO DYC

Entrada: x

si *esPequeno*(x)

 devolver *solucion_trivial*(x)

sino

$\langle x_1, \dots, x_n \rangle \leftarrow \text{descomponer}(x)$

 para $i = 1$ hasta n

$y_i \leftarrow \text{DYC}(x_i)$

 fin para

 devolver *combinar*($y_1, \dots, y_n$)

fin si

Nota: se debe entender que el esquema anterior es genérico y en cada problema pueden no aparecer todos los métodos descriptos.



Problema Búsqueda Binaria





Búsqueda Binaria

El problema de la búsqueda binaria consiste en buscar un número x dado en una secuencia ordenada en forma creciente de números S que también es un dato de entrada.

Como resultado del problema se deberá devolver “*verdadero*” en el caso de que el x pertenezca a S , y “*falso*” en caso contrario.

Búsqueda Binaria – Estrategia de Resolución

Dada la naturaleza del problema se puede diseñar una resolución por Divide y Conquista siguiendo la siguiente estrategia:

Caso Base

1. Si la longitud de **S** es UNO se puede determinar unívocamente si **x** pertenece a **S** o no.
2. Si tomamos el valor ubicado a la mitad de **S**, verificamos si es igual a **x** ya tenemos el resultado.

Subproblemas

1. Si **x** es mayor al valor ubicado en la mitad de **S** buscamos en la mitad derecha ubicada entre $(n/2)+1$ y **n**
2. Si **x** es menor al valor ubicado en la mitad de **S** buscamos en la mitad izquierda ubicada entre 1 y $(n/2)-1$

Combinación

1. En este caso en particular se deberá devolver en forma directa el resultado del subproblema evaluado dado que solo uno de ellos se ejecutará.

Búsqueda Binaria – Ejemplo

	2	3	5	7	8	9	12	15	17
índice	1	2	3	4	5	6	7	8	9

$x = 12$

Si obtenemos la mitad, sería $(9+1)/2 = 5$. Si evaluamos el valor en la posición 5 vemos que es 8, por lo cual no es igual al 12 que buscamos. Como ese valor de la mitad es menor a 12 se busca hacia la parte derecha:

	2	3	5	7	8	9	12	15	17
índice	1	2	3	4	5	6	7	8	9

$x = 12$

Si obtenemos la mitad a partir de la parte derecha considerada que iría entre los índices 6 y 9, sería $(9+6)/2 = 7$ (considerando la parte entera inferior). Si evaluamos el valor en la posición 7 vemos que es 12, por lo cual es igual al 12 que buscamos y ya podríamos devolver “verdadero”.

Búsqueda Binaria – Algoritmo

ALGORITMO BUSQUEDABIN

Entrada: S : Vector<entero>, x : entero

Salida: verdadero o falso

si $longitud(S) = 1$

devolver $S[0] = x$

sino

$y \leftarrow S[longitud(S)/2]$

si $x = y$

devolver verdadero

sino

$mitad \leftarrow longitud(S)/2$

si $x < y$

devolver $BusquedaBin(S[0, mitad - 1], x)$

sino

devolver $BusquedaBin(S[mitad, longitud(S) - 1], x)$

fin si

fin si

fin si

Búsqueda Binaria – Complejidad Temporal

ALGORITMO BUSQUEDABIN

Entrada: S : Vector<entero>, x : entero

Salida: verdadero o falso

si $longitud(S) = 1$
 devolver $S[0] = x$

sino

$y \leftarrow S[longitud(S)/2]$

 si $x = y$

 devolver verdadero

 sino

$mitad \leftarrow longitud(S)/2$

 si $x < y$

 devolver $BusquedaBin(S[0, mitad - 1], x)$

 sino

 devolver $BusquedaBin(S[mitad, longitud(S) - 1], x)$

 fin si

fin si

fin si

$$T(n) = \begin{cases} c & \text{si } n = 1 \\ 1T(n/2) + c & \text{si } n > 1 \end{cases}$$

$a = 1$, $b = 2$ y $k = 0$, entonces

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$

$$\Theta(n^0 \log(n)) = \Theta(\log(n))$$



Problema de una palabra Palíndromo



Determinar si una palabra Palíndromo

- Una palabra se dice que es Palíndromo si se lee igual en un sentido que en otro, es decir, si se mira de izquierda a derecha es igual que de derecha a izquierda. Si se trata de números en lugar de letras, se llama capicúa.
- Ejemplos de palabras que cumplen la definición de Palíndromo son:
Ana, radar, reconocer, rotor, salas, somos, etc.
- Lo que se requiere es una estrategia de solución que a partir de una palabra dada determine si la misma es Palíndromo o no.

Determinar si una palabra Palíndromo - Estrategia

- La estrategia de resolución del problema por Divide y Conquista se puede describir de la siguiente manera:

Caso Base

Si la secuencia tiene longitud cero o uno, la solución trivial es devolver verdadero dado que la secuencia se considera palíndromo.

Subproblemas

Comparar los extremos de la secuencia, si no coinciden se devuelve falso, y sino se reduce la secuencia en dos extremos, y se resuelve si es palíndromo esa secuencia menor.

Combinación

Devolver el resultado de la secuencia de $n-2$.

Determinar si una palabra Palíndromo - Estrategia

- A partir de la estrategia anterior, a partir de un ejemplo podemos validar la misma:

Ejemplo “Verdadero”

s	a	l	a	s
---	---	---	---	---

s	a	l	a	s
---	---	---	---	---

Suproblema 1:

a	l	a
---	---	---

a	l	a
---	---	---

Suproblema 2 (sería el caso base) :

l

 → Devuelve “Verdadero”

Ejemplo “Falso”

s	o	l	a	s
---	---	---	---	---

s	o	l	a	s
---	---	---	---	---

Suproblema 1:

o	l	a
---	---	---



Al comparar los extremos son distintos, entonces devuelve “Falso”

Determinar si una palabra Palíndromo - Algoritmo

ALGORITMO EsPalindromo

Entrada: S: secuencia o palabra sobre la cual determinar si es palíndromo o no; **inicio**, **fin**: índices a trabajar sobre la secuencia

Salida: verdadero o falso si la secuencia es Palíndromo o no

```
sí (inicio >= fin)
    devolver verdadero
fin si
sino
    sí S[inicio] <> S[fin]
        devolver falso
    fin si
    sino
        devolver EsPalindromo (S, inicio+1, fin-1)
    fin sino
fin sino
fin EsPalindromo
```

Palabra Palíndromo – Complejidad Temporal

ALGORITMO ESPalindromo

Entrada: S: secuencia o palabra sobre la cual determinar si es palíndromo o no; **inicio**, **fin**: índices a trabajar sobre la secuencia

Salida: verdadero o falso si la secuencia es Palíndromo o no

Es un caso de resta:

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$

$$a=1 ; b = 2; k=0$$

```
    sí (inicio >= fin)
        devolver verdadero
    fin si
    sino
        sí S[inicio] <> S[fin]
            devolver falso
        fin si
        sino
            devolver ESPalindromo (S, inicio+1, fin-1)
        fin sino
    fin sino
fin ESPalindromo
```

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

Como $a = 1$ entonces la complejidad temporal es:

$$\mathcal{O}(n)$$



Función de Fibonacci



Función de Fibonacci

La función de Fibonacci se define con la siguiente función:

$$F(n) = \begin{cases} 1 & \text{si } n < 2 \\ F(n-1) + F(n-2) & \text{sino} \end{cases}$$

En donde el valor de la función es 1 para $n < 2$, y sino para un $n \geq 2$ se define como la suma de *Fibonacci* ($n-1$) + *Fibonacci* ($n-2$).

Función de Fibonacci - Estrategia

La estrategia por divide y conquista para la función de Fibonacci se puede describir como:

Caso base

Por definición de la función si $n < 2$ se devuelve 1(unos)

Subproblemas

Por definición de la función, el problema general se divide en dos problemas menores, uno con $n-1$ y otro con $n-2$

Combinación

Sumatoria del resultado de $n-1$ y de $n-2$

Función de Fibonacci - Algoritmo

ALGORITMO FIBONACCI

Entrada: n: número para el cálculo de la función

Salida: valor correspondiente al cálculo de Fibonacci

sí ($n < 2$)

devolver 1

fin si

sino

devolver Fibonacci($n-1$) + Fibonacci($n-2$)

fin sino

fin Fibonacci

Función de Fibonacci – Complejidad Temporal

ALGORITMO FIBONACCI

Entrada: n: número para el cálculo de la función

Salida: valor correspondiente al cálculo de Fibonacci

```
sí (n<2)
    devolver 1
fin si
sino
    devolver Fibonacci(n-1) + Fibonacci(n-2)
fin sino
fin Fibonacci
```

Caso Resta:

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$

$$a=2 ; b = 1; k=0$$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \text{ div } b}) & \text{si } a > 1 \end{cases}$$

Por las opciones de resolución de las funciones se considera $a > 1$, entonces nos queda:

$$\Theta(2^n)$$

Función de Fibonacci – Análisis resolución

- ¿Es una buena solución la obtenida para resolver Fibonacci utilizando la técnica divide y conquista?
Si analizamos la complejidad temporal se podría decir que no.
- Uno de los puntos que no favorece la solución dada es que un mismo valor se calcula varias veces, sin poder ser reutilizado.
- Esos valores que se repiten se pueden ver en la siguiente secuencia, en dónde **$F(3)$** o **$F(2)$** se ven más de una vez, lo mismo sucedería con **$F(1)$** si se sigue la apertura del cálculo:

$$F(5) = \underbrace{F(4) + F(3)}_{F(5)} = \underbrace{F(3) + F(2)}_{F(4)} + \underbrace{F(2) + F(1)}_{F(3)} =$$



Métodos de Ordenamiento



Métodos de Ordenamiento

- Existen métodos de ordenamiento que denominaremos “tradicionales”, en los cuales con alguna estrategia iterativa se puede llegar al ordenamiento, como son los siguientes:

SELECCIÓN

Consiste en buscar en toda la secuencia de elementos el menor y ponerlo en la posición 1, luego de los restantes el menor en la posición 2, y así hasta llegar a ordenar todos los elementos.

INSERCIÓN

Consiste en ir ubicando cada elemento en su posición. Es decir que en cada iteración i se obtiene una secuencia ordenada de $i-1$ elementos en la primera parte del arreglo y se inserta el elemento de la posición i .

Métodos de Ordenamiento - Selección

ALGORITMO SELECCION

Entrada: S : Vector<entero>

$n \leftarrow longitud(S)$

para $i = 0$ hasta $n - 1$

$m \leftarrow i$

para $j = i + 1$ hasta $n - 1$

si $S[j] < S[m]$

$m \leftarrow j$

fin si

fin para

$aux \leftarrow S[i];$

$S[i] \leftarrow S[m];$

$S[m] \leftarrow aux;$

fin para

Un ciclo que itera n veces en el peor de los casos, dentro de otro ciclo que itera n veces, nos da una complejidad de:

$$\mathcal{O}(n^2).$$

Métodos de Ordenamiento - Inserción

ALGORITMO INSERCION

Entrada: S : Vector<entero>

$n \leftarrow longitud(S)$

para $i = 1$ hasta $n - 1$

 para $j = i$ hasta 1 decrementando 1

 si $S[j] < S[j - 1]$

$aux \leftarrow S[j - 1];$

$S[j - 1] \leftarrow S[j];$

$S[j] \leftarrow aux;$

 fin si

 fin para

fin para

6	2	9	7	11	3
---	---	---	---	----	---

2	6	9	7	11	3
---	---	---	---	----	---

2	6	9	7	11	3
---	---	---	---	----	---

2	6	7	9	11	3
---	---	---	---	----	---

2	6	7	9	11	3
---	---	---	---	----	---

2	6	7	9	11	3
---	---	---	---	----	---

2	6	7	9	3	11
---	---	---	---	---	----

2	6	7	3	9	11
---	---	---	---	---	----

2	6	3	7	9	11
---	---	---	---	---	----

2	3	6	7	9	11
---	---	---	---	---	----

Métodos de Ordenamiento - Inserción

ALGORITMO INSERCIÓN

Entrada: S : Vector<entero>

$n \leftarrow longitud(S)$

para $i = 1$ hasta $n - 1$

 para $j = i$ hasta 1 decrementando 1

 si $S[j] < S[j - 1]$

$aux \leftarrow S[j - 1];$

$S[j - 1] \leftarrow S[j];$

$S[j] \leftarrow aux;$

 fin si

 fin para

fin para

Un ciclo que itera n veces en el peor de los casos, dentro de otro ciclo que itera n veces, nos da una complejidad de:

$$\mathcal{O}(n^2).$$

Métodos de Ordenamiento – Divide y Conquista

- Dentro de los métodos de ordenamiento por Divide y Conquista, se van a estudiar dos algoritmos que son los siguientes:
 - a. Merge-Sort*
 - b. Quick-Sort*
- Cada uno de los algoritmos será analizado en cuanto a su estrategia y complejidad temporal.

Nota: Por ordenamiento se entiende “ordenar” una secuencia en forma creciente.



Merge Sort



Merge Sort – Estrategia

La estrategia por Divide y Conquista del Merge Sort se puede definir como sigue:

Caso Base

Una secuencia de un solo elemento, dado que ya por defecto se considera ordenada.

Subproblemas

Dividir la secuencia de entrada en dos mitades, y ordenar cada una de las mitades por separado

Combinación

Se deben mezclar las dos mitades ordenadas en una nueva secuencia ordenada, lo cual dará como resultado la secuencia total ordenada.

Merge Sort – Ejemplo usando la Estrategia

6	3	11	15	4	7	9	2
---	---	----	----	---	---	---	---

6	3	11	15
---	---	----	----

4	7	9	2
---	---	---	---

6	3
---	---

11	15
----	----

4	7
---	---

9	2
---	---

6

3

11

15

4

7

9

2

Casos Bases

3	6
---	---

11	15
----	----

4	7
---	---

2	9
---	---

Se coloca el menor, y se avanza en la primera parte, y se repite el proceso

3	6	11	15
---	---	----	----

2	4	7	9
---	---	---	---

2	3	4	6	7	9	11	15
---	---	---	---	---	---	----	----

Comienzo a combinar las dos subsecuencias, comparando el primero de cada una, el menor va al resultado, y así avanza en la parte del cual se tomo el valor, y nuevamente se compara.

Merge Sort – Algoritmo

ALGORITMO MERGESORT

Entrada: S : Vector<entero>, $inicio$: entero, fin : entero

si $inicio < fin$

entero $medio \leftarrow (fin + inicio)/2$

$MergeSort(S, inicio, medio)$

$MergeSort(S, medio + 1, fin)$

$Merge(S, inicio, fin)$

fin si

Nota: como se puede observar la combinación de las dos partes ordenadas se hace en un algoritmo adicional, el cual se describirá a continuación.

Merge Sort – Algoritmo

ALGORITMO MERGE

Entrada: S : Vector<entero>, $inicio$: entero, fin : entero

Vector $R \leftarrow inicializarVector(fin - inicio + 1)$

entero $medio \leftarrow (inicio + fin)/2$

$i \leftarrow inicio$

$j \leftarrow medio + 1$

para entero $k = 0$ hasta $fin - inicio$

si $(j > fin \text{ O } S[i] \leq S[j]) \text{ Y } i \leq medio$

$R[k] \leftarrow S[i]$

$i \leftarrow i + 1$

sino

$R[k] \leftarrow S[j]$

$j \leftarrow j + 1$

fin si

fin para

para entero $k = 0$ hasta $fin - inicio$

$S[inicio + k] \leftarrow R[k]$

fin para

Merge Sort – Complejidad Temporal

ALGORITMO MERGESORT

Entrada: S : Vector<entero>, $inicio$: entero, fin : entero

si $inicio < fin$

entero $medio \leftarrow (fin + inicio)/2$

$MergeSort(S, inicio, medio)$

$MergeSort(S, medio + 1, fin)$

$Merge(S, inicio, fin)$

fin si

Existen dos llamados recursivos en donde por cada parte el vector se divide en dos, por lo cual:

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2T(n/2) + P^1(n) & \text{si } n > 1 \end{cases} \quad a = 2, b = 2$$

Debemos analizar la complejidad de $Merge$ para determinar el k .

Merge Sort – Complejidad Temporal

ALGORITMO MERGE

Entrada: S : Vector<entero>, $inicio$: entero, fin : entero

Vector $R \leftarrow inicializarVector(fin - inicio + 1)$

entero $medio \leftarrow (inicio + fin)/2$

$i \leftarrow inicio$

$j \leftarrow medio + 1$

para entero $k = 0$ hasta $fin - inicio$

si $(j > fin \text{ O } S[i] \leq S[j]) \text{ Y } i \leq medio$

$R[k] \leftarrow S[i]$

$i \leftarrow i + 1$

sino

$R[k] \leftarrow S[j]$

$j \leftarrow j + 1$

fin si

fin para

para entero $k = 0$ hasta $fin - inicio$

$S[inicio + k] \leftarrow R[k]$

fin para

La complejidad esta dada por un ciclo en el peor de los casos de n , por lo tanto:

$$\mathcal{O}(n)$$

Merge Sort – Complejidad temporal

ALGORITMO MERGESORT

Entrada: S : Vector<entero>, $inicio$: entero, fin : entero

si $inicio < fin$

entero $medio \leftarrow (fin + inicio)/2$

$MergeSort(S, inicio, medio)$

$MergeSort(S, medio + 1, fin)$

$Merge(S, inicio, fin)$

fin si

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2T(n/2) + P^1(n) & \text{si } n > 1 \end{cases} \quad a = 2, \bar{b} = 2 \text{ y } k = 1,$$

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log(n)) & \text{si } a = b^k \\ \Theta(n^{\log_b(a)}) & \text{si } a > b^k \end{cases}$$



$$\Theta(n \log(n))$$



Quick Sort





Quick Sort – Definición

- La estrategia del Quick Sort consiste en seleccionar un elemento al azar denominado *Pivot*.
- A partir de ese Pivot se divide la secuencia en dos partes, a la izquierda del pivot todos los elementos menores y a la derecha del pivot todos los elementos mayores al mismo.
- Una vez hecho esto, el pivot estará ubicado en su posición definitiva en la secuencia y se aplicara en forma recursiva el método en las dos partes resultantes.



Quick Sort – Pivot

- Existen distintas técnicas para seleccionar el pivot, algunas mas sencillas y otras más complejas. El criterio seleccionada deberá redundar en una solución más eficiente.
- Cuanto mas cercano al elemento mediano se encuentre el pivot, mas parejas serán las dos partes de la secuencia y mejor la solución, cuanto mas cercano a alguno de los extremos se encuentre el pivot, más desparejas serán las dos partes y menos eficiente la solución.
- Un criterio simple es considerar como pivot el primer elemento de la secuencia inicial.

Quick Sort – Estrategia

- La estrategia de resolución por Divide y Conquista se puede enumerar como sigue:

Caso Base

Una secuencia que tenga una o dos posiciones. En nuestro caso se va a considerar una sola posición.

Subproblemas

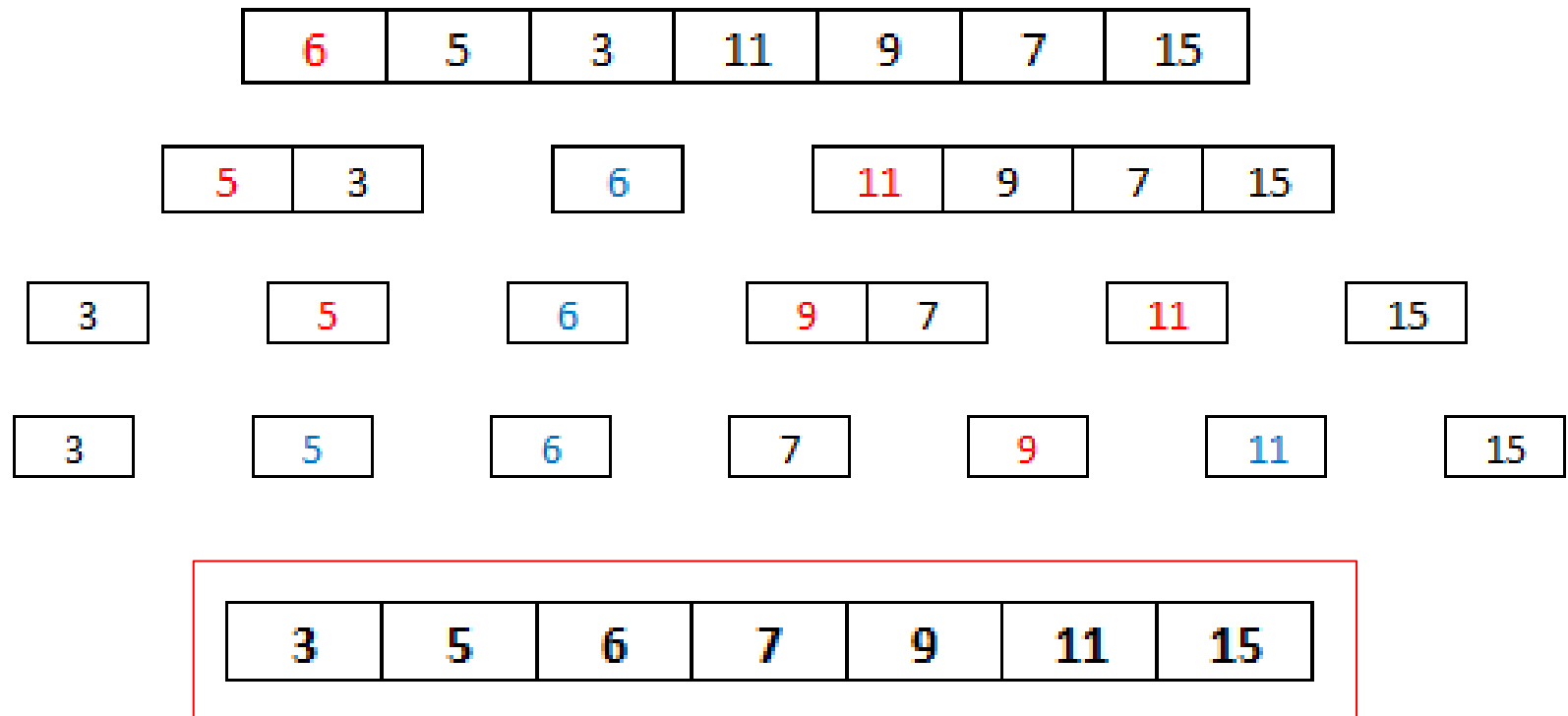
Dividir en dos subproblemas en base al Pivot, enviando a ordenar cada una de esas dos partes.

Combinación

No se requiere.

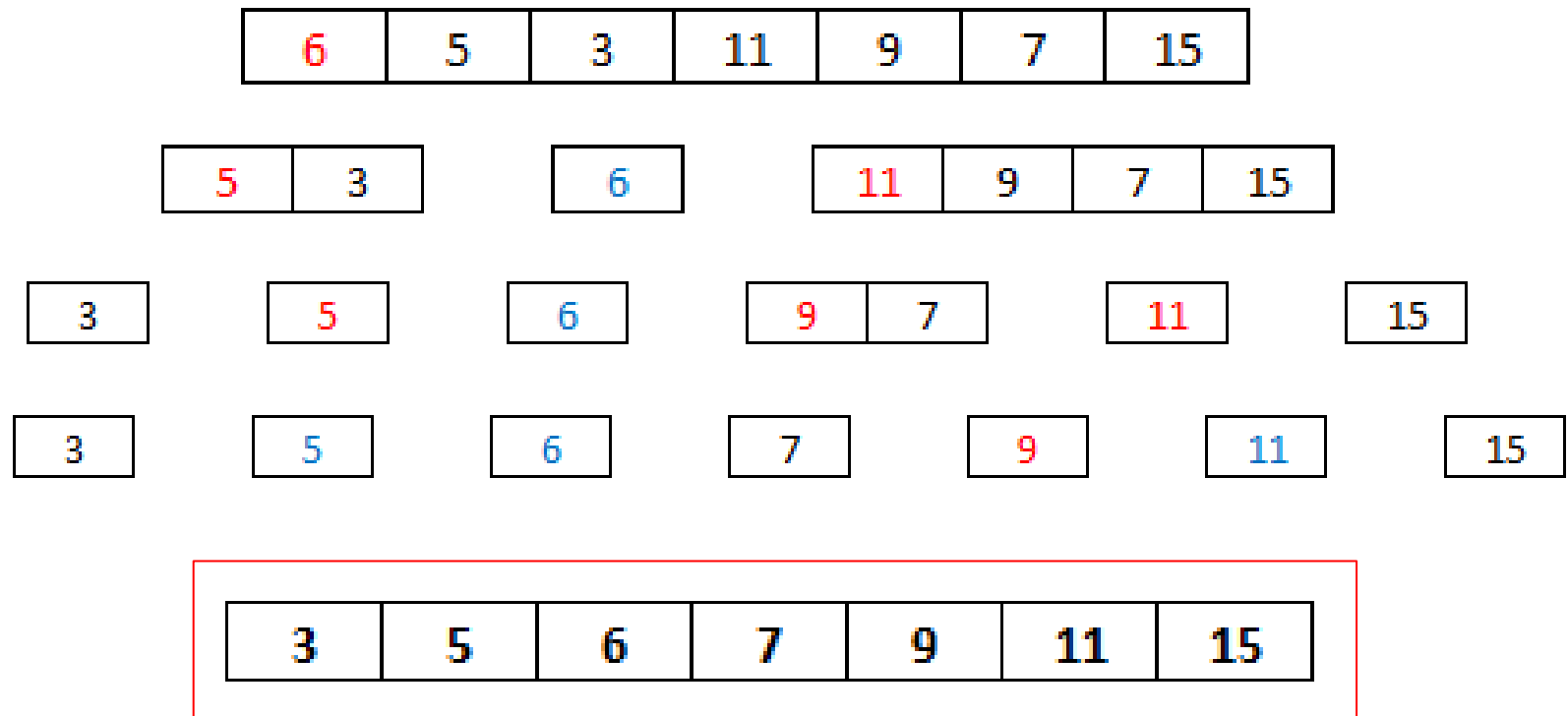
Quick Sort – Ejemplo

A partir del siguiente ejemplo se mostrará la estrategia del Quick Sort definida. En rojo se marca el elemento seleccionado como Pivot en cada iteración, y en azul como queda ubicado luego:



Quick Sort – Ejemplo

A partir del siguiente ejemplo se mostrará la estrategia del Quick Sort definida. En rojo se marca el elemento seleccionado como Pivot en cada iteración, y en azul como queda ubicado luego:



Quick Sort – Algoritmo

ALGORITMO QUICKSORT

Entrada: S : Vector<entero>, $inicio$:entero, fin :entero

si $inicio < fin$

$p \leftarrow pivot(S, inicio, fin)$

$QuickSort(S, inicio, p - 1)$

$QuickSort(S, p + 1, fin)$

fin si

Nota: como se puede observar el método *Pivot* se realiza en forma externa, y como resultado devuelve la posición en donde se encuentra ese elemento **Pivot** luego de haber ubicado a su izquierda los menores y a su derecha los mayores. A continuación se muestra el algoritmo del mismo.

Quick Sort – Algoritmo

ALGORITMO PIVOT

Entrada: S : Vector<entero>, $inicio$:entero, fin :entero

Salida: p : entero

```
entero  $p \leftarrow S[inicio]$ 
entero  $k \leftarrow inicio + 1$ 
entero  $l \leftarrow fin$ 
mientras  $S[k] \leq p \text{ Y } k < fin$ 
     $k \leftarrow k + 1$ 
fin mientras
mientras  $S[l] > p$ 
     $l \leftarrow l - 1$ 
fin mientras
mientras  $k < l$ 
     $aux \leftarrow S[k]$ 
     $S[k] \leftarrow S[l]$ 
     $S[l] \leftarrow aux$ 
    mientras  $S[k] \leq p$ 
         $k \leftarrow k + 1$ 
    fin mientras
    mientras  $S[l] > p$ 
         $l \leftarrow l - 1$ 
    fin mientras
fin mientras
 $aux \leftarrow S[inicio]$ 
 $S[inicio] \leftarrow S[l]$ 
 $S[l] \leftarrow aux$ 
devolver  $l$ 
```

Quick Sort – Complejidad Temporal

ALGORITMO QUICKSORT

Entrada: S : Vector<entero>, $inicio$:entero, fin :entero

si $inicio < fin$

$p \leftarrow pivot(S, inicio, fin)$

$QuickSort(S, inicio, p - 1)$

$QuickSort(S, p + 1, fin)$

fin si

Debido a que el pivot no se sabe si queda a la mitad o en un extremo se analizan las dos posibles situaciones, caso promedio que divida a la mitad, y peor caso que quede en alguno de los extremos.

Caso Promedio

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2T(n/2) + P^1(n) & \text{si } n > 1 \end{cases}$$

$$a = 2, \bar{b} = 2$$

Peor Caso

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ T(n - 1) + P^1(n) & \text{si } n > 1 \end{cases}$$

$$a = 1, b = 1$$

En ambos casos se debe analizar la complejidad de *Pivot*

Quick Sort – Complejidad Temporal

ALGORITMO PIVOT

Entrada: S : Vector<entero>, $inicio$:entero, fin :entero

Salida: p : entero

```
entero  $p \leftarrow S[inicio]$   
entero  $k \leftarrow inicio + 1$   
entero  $l \leftarrow fin$ 
```

```
mientras  $S[k] \leq p \text{ Y } k < fin$   
   $k \leftarrow k + 1$ 
```

```
fin mientras
```

```
mientras  $S[l] > p$   
   $l \leftarrow l - 1$ 
```

```
fin mientras
```

```
mientras  $k < l$ 
```

```
   $aux \leftarrow S[k]$   
   $S[k] \leftarrow S[l]$   
   $S[l] \leftarrow aux$ 
```

```
  mientras  $S[k] \leq p$   
     $k \leftarrow k + 1$ 
```

```
  fin mientras
```

```
  mientras  $S[l] > p$   
     $l \leftarrow l - 1$ 
```

```
  fin mientras
```

```
fin mientras  
 $aux \leftarrow S[inicio]$   
 $S[inicio] \leftarrow S[l]$   
 $S[l] \leftarrow aux$   
devolver  $l$ 
```

La complejidad esta dado por diferentes ciclos en el peor de los casos esos ciclos iteran n veces. Los ciclos anidados por la combinación de índices no iteran más que n veces en conjunto, por lo tanto

$$O(n)$$

Quick Sort – Complejidad Temporal

ALGORITMO QUICKSORT

Entrada: S : Vector<entero>, $inicio$:entero, fin :entero

si $inicio < fin$

$p \leftarrow pivot(S, inicio, fin)$

$\mathcal{O}(n)$

QuickSort($S, inicio, p - 1$)

QuickSort($S, p + 1, fin$)

fin si

Caso Promedio

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 2T(n/2) + P^1(n) & \text{si } n > 1 \end{cases}$$

$$a = 2, \bar{b} = 2 \text{ y } k = 1,$$

$$\Theta(n \log(n))$$

Peor Caso

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ T(n - 1) + P^1(n) & \text{si } n > 1 \end{cases}$$

$$a = 1, b = 1 \text{ y } k = 1,$$

$$\Theta(n^2)$$



¡Muchas gracias!

